BUSINESS REPORT

Executive Summary

The Report describes the development of a supervised machine learning model for predicting game ratings. The model is based on a combination of Linear Regression, Decision Trees, and Random Forests. This is a powerful machine learning technique that can be used to predict game ratings at very high accuracy. It can process numerical data as well as categorical data without intensive preprocessing. Its robustness in handling various input features and their interactions greatly enhances its predictive power. The results of this study show that the developed model can be a robust and scalable solution for predicting gaming ratings, hence escalating PQC with improved service offerings and advantages over the competition in the online gaming community.

Table of Content

Introduction 3
1. Business Understandings 3
Business Problem 3
BACCM Elements 4
2. Data Understanding, Preparation, Exploration, Visualization, and Insights 4
Data Understanding 4
Data Preparation 5
Data Exploration and Visualization 5
Insights Gained 16
3. The Machine Learning Approach 17
Model Selection 17
Model Development 17
Performance Metrics 18
4. The Model and Performance Metrics 18
Model Description 18
Performance Evaluation 19
5. Discussion of the Model: Pros and Cons 19
Pros 19
Cons 20
6. Business solutions and recommendations 20
Implementation 20
Recommendations 21
Conclusion 21
Appendices 22
Appendix 1: Count Plot of Game_Type 22
Appendix 2: Scatter Plot between Actual values vs Predicted values 22
Appendix 3: Histogram of Predicted values 23

Introduction

Play Quest Conquer, shortly called PQC, is an online gaming platform based in Sydney, Australia, but with operations stretching out to the global market. It offers subscription services whereby users log into the website, view the various online games available, purchase any game of interest, and add it to their personal collection for playing purposes or inviting friends and even trading among players. The key PQC business objective is understanding what factors influence game ratings, which will eventually enhance user satisfaction and engagement. From that point of view, this consulting project aims to develop a supervised machine learning model for predicting game ratings based on the dataset provided by the market research team across PQC.

1. Business Understandings

Business Problem

Core business problem PQC looks to solve the key drivers of game ratings. Understanding these variables will inform strategic decisions in game development, acquisition, and marketing for improved user satisfaction and higher engagement on the platform. If PQC can predict game ratings accurately, it can shift focus to developing and promoting games that are more likely to have a good reception from users, hence optimizing resource deployment and boosting overall user satisfaction.

BACCM Elements

The applicable elements of the Business Analysis Core Concept Model in this case study are Change, Need, Solution, Stakeholders, Value, and Context. Change means implementing data-driven strategies to achieve higher game ratings and satisfy more users. The Need is to understand the drivers of game ratings. The solution includes the development of a supervised machine-learning model for predicting game ratings.

The main stakeholders would be the market research team, game developers, marketing team, and users at PQC. Value comes in the form of enhanced user satisfaction, acquisition of more games, and better resource allocation. Context applies to the online gaming industry by examining users' preferences and engagement metrics.

2. Data Understanding, Preparation, Exploration, Visualization, and Insights

Data Understanding

PQC's market research team provided the dataset with attributes relevant to predicting game ratings. Among the incorporated attributes are game types, year of release, age category, minimum and maximum numbers of players, playtime, and ratings. Average Rating and Adjusted Average Rating are also differentiated herein; the latter case rate adjustment will be based on the

number of raters to provide fairness in each rating. Understanding these characteristics is very important for effective data preprocessing and analysis.

Data Preparation

Data preparation means cleaning the dataset for suitability in analysis. Handling missing values, outliers, and encoding categorical variables are part of data preparation . Missing values may either be imputed or removed, considering their importance and the effect on the analysis of the data. Outliers are those that may cause distortion to model performance; hence, they need identification and handling accordingly. Categorical variables, such as game type and age category, are engineered using one-hot encoding or similar methods to enhance their utilization in machine learning models. Standardization of numeric features provides equal contribution from all variables during model learning.

Data Exploration and Visualization

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import date
```

```
df = pd.read_csv("PQC_data.csv")
```

```
df.head()
```

	Game_ID	Game_Name	Released_Year	Game_Type	Age_Category	Min_Players	Max_Players	Average_Complexity	Complexity_Raters	Average_Play_Time	Owner_Number	Trader_Number
0	9048	Age of Industry	2010	BaseGame	12 to under 18	2	5	3.3636	231	120	2697	71
1	40086	Going The Distance	1998	BaseGame	under 5	2	8	2.0000	3	60	3	0
2	12545	Inside	2007	BaseGame	5 to under 12	2	2	2.0000	12	10	187	8
3	62639	New Bedford	2016	BaseGame	12 to under 18	1	4	2.4000	5	75	62	1
4	26419	Sekigahara fierce fight	1988	BaseGame	12 to under 18	1	2	3.0000	2	120	4	0

Figure 1: Importing libraries to interpret the dataset

(Source: Self-Created)

The image shows Python code importing libraries like Pandas, NumPy, and Matplotlib for data manipulation and visualization. It then reads a CSV file named "PQC_data.csv" into a pandas Data Frame and displays the first few rows using the head() method.

```
df.shape
```

```
(24813, 17)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24813 entries, 0 to 24812
Data columns (total 17 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Game_ID               24813 non-null  int64
 1   Game_Name             24806 non-null  object
 2   Released_Year         24813 non-null  int64
 3   Game_Type             24813 non-null  object
 4   Age_Category          24813 non-null  object
 5   Min_Players           24813 non-null  int64
 6   Max_Players           24813 non-null  int64
 7   Average_Complexity    24813 non-null  float64
 8   Complexity_Raters     24813 non-null  int64
 9   Average_Play_Time     24813 non-null  int64
 10  Owner_Number          24813 non-null  int64
 11  Trader_Number         24813 non-null  int64
 12  HighInterest_Number   24813 non-null  int64
 13  Interest_Number       24813 non-null  int64
 14  Rater_Number          24813 non-null  int64
 15  Comment_Number        24813 non-null  int64
 16  Average_Rating        24813 non-null  float64
dtypes: float64(2), int64(12), object(3)
memory usage: 3.2+ MB
```

Figure 2: Data Frame Creation

(Source: Self-Made)

The image shows the shape and information of a Pandas Data Frame. It has 24813 rows and 17 columns with data types like integers, floats, and objects.

```
df.isnull().sum()
```

```
Game_ID                0
Game_Name              7
Released_Year          0
Game_Type              0
Age_Category           0
Min_Players            0
Max_Players            0
Average_Complexity     0
Complexity_Raters      0
Average_Play_Time      0
Owner_Number           0
Trader_Number          0
HighInterest_Number    0
Interest_Number        0
Rater_Number           0
Comment_Number         0
Average_Rating         0
dtype: int64
```

```
df[df.isnull().any(axis=1)]
```

	Game_ID	Game_Name	Released_Year	Game_Type	Age_Category	Min_Players	Max_Players	Average_Complexity	Complexity_Raters	Average_Play_Time	Owner_Number	Trader_Number
2586	54895	NaN	2012	BaseGame	5 to under 12	2	2	4.0000	1	30	16	1
5854	50662	NaN	2011	BaseGame	5 to under 12	2	5	2.5000	2	40	12	0
6781	41160	NaN	1988	BaseGame	5 to under 12	2	17	2.0000	3	90	30	0
9222	57866	NaN	1989	BaseGame	5 to under 12	2	4	1.0000	1	30	0	0
9522	59871	NaN	2011	BaseGame	12 to under 18	4	7	1.3333	3	30	24	0
16479	15762	NaN	2006	BaseGame	5 to under 12	2	8	1.0000	4	30	74	0
23938	41161	NaN	1989	BaseGame	5 to under 12	2	6	2.0000	2	180	2	0

Figure 3: Missing values in DataFrame

(Source: Own-Creation)

The image shows Python code output related to missing values in a DataFrame. It counts missing values per column and displays rows with at least one missing value, revealing issues with the 'Game_Name' and 'Game_Type' columns.

```
df = df.dropna()
```

```
df.isnull().sum()
```

```
Game_ID                0
Game_Name              0
Released_Year          0
Game_Type              0
Age_Category           0
Min_Players            0
Max_Players            0
Average_Complexity     0
Complexity_Raters      0
Average_Play_Time      0
Owner_Number           0
Trader_Number          0
HighInterest_Number    0
Interest_Number        0
Rater_Number           0
Comment_Number         0
Average_Rating         0
dtype: int64
```

```
df.shape
```

```
(24806, 17)
```

```
df.describe()
```

	Game_ID	Released_Year	Min_Players	Max_Players	Average_Complexity	Complexity_Raters	Average_Play_Time	Owner_Number	Trader_Number	HighInterest_Number	Interest_Number	Rater_Number	Comment_Nur
count	24806.000000	24806.000000	24806.000000	24806.000000	24806.000000	24806.000000	24806.000000	24806.000000	24806.000000	24806.000000	24806.000000	24806.000000	24806.00
mean	30931.797146	1997.200274	2.061921	5.320890	1.985793	42.565670	55.212287	674.387890	23.193784	32.469886	111.213456	426.929533	126.02
std	18958.583905	126.105276	0.674197	7.077461	0.825667	190.131437	42.584591	2453.831783	61.481204	95.838770	378.802156	1881.445405	457.68
min	7074.000000	-99.000000	0.000000	0.000000	1.000000	1.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00
25%	14649.500000	2000.000000	2.000000	4.000000	1.230800	1.000000	25.000000	17.000000	0.000000	0.000000	2.000000	7.000000	4.00
50%	24711.000000	2007.000000	2.000000	4.000000	2.000000	4.000000	45.000000	77.000000	3.000000	3.000000	9.000000	27.000000	13.00
75%	47157.250000	2012.000000	2.000000	6.000000	2.500000	16.000000	60.000000	376.000000	17.000000	18.000000	49.000000	142.000000	57.00
max	71822.000000	2017.000000	9.000000	99.000000	5.000000	5996.000000	180.000000	73188.000000	1395.000000	1586.000000	6402.000000	53680.000000	11798.00

Figure 4: Descriptive statistics of the DataFrame

(Soure: Self-Developed)

The image shows Python code output related to missing values and descriptive statistics of a DataFrame. It reveals missing values in the 'Game Name' and 'Game Type' columns and provides summary statistics for numerical columns like count, mean, standard deviation, and percentiles.

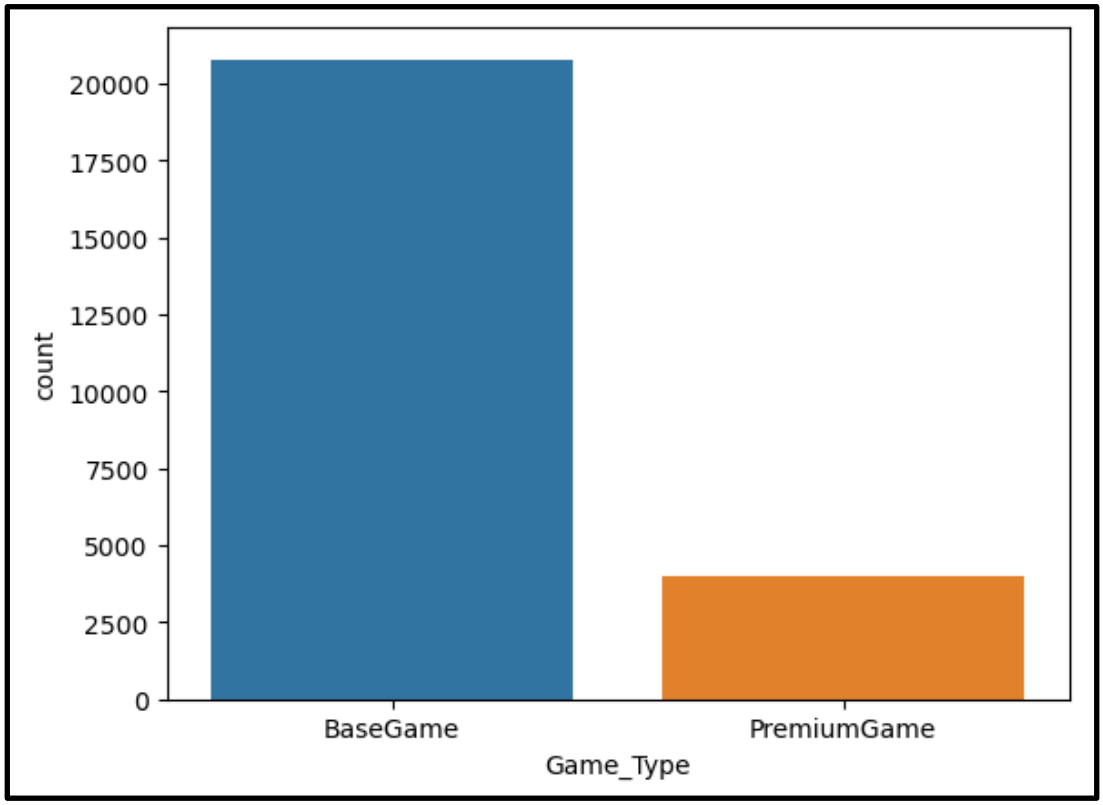


Figure 5: Count Plot of Game_Type

(Source: Self-Made)

The image shows a count plot of the column 'Game_Type' which has two unique values: 'BaseGame' and 'PremiumGame'. The count plot shows that the data is imbalanced, with many more instances of 'BaseGame' than 'PremiumGame'.

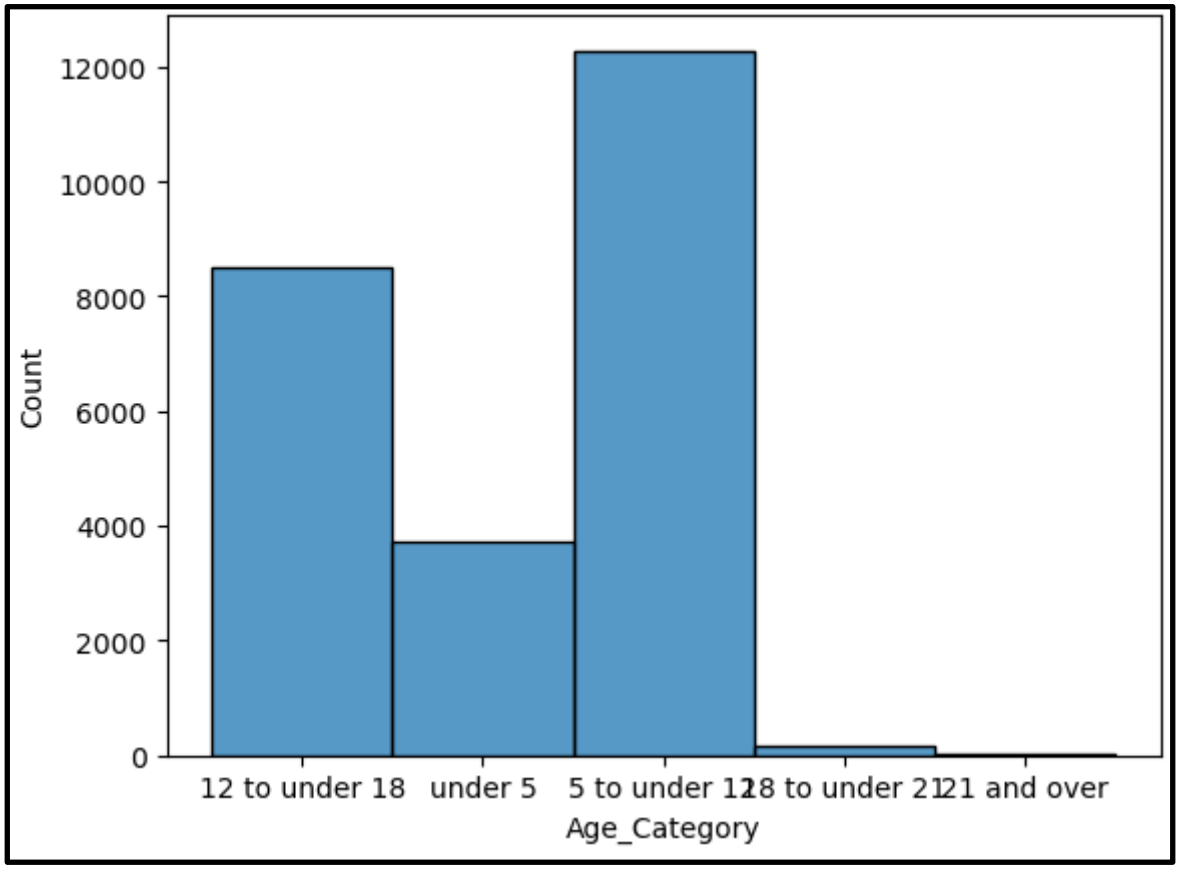


Figure 6: Histogram of Age_Category

(Source: Self-Developed)

The image shows a histogram of the 'Age_Category' column. It reveals that the most frequent age category is '12 to under 18', followed by '5 to under 12'. The other categories have significantly fewer occurrences.

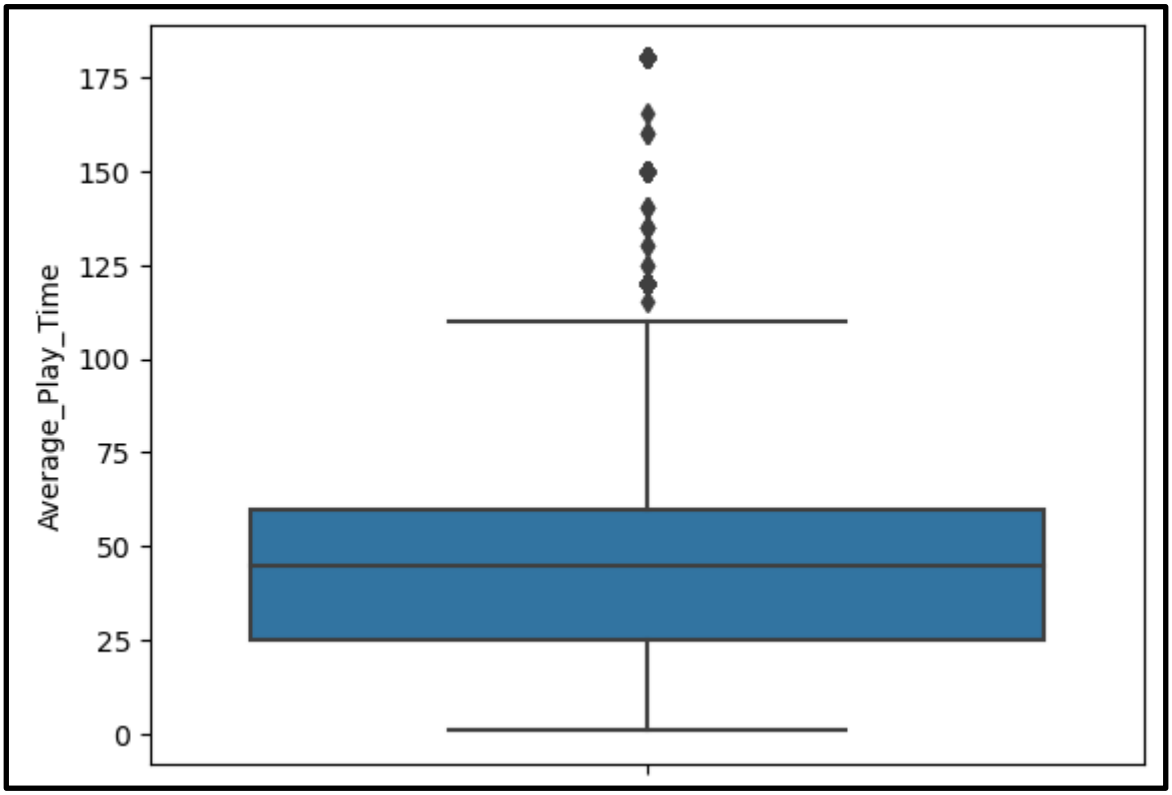


Figure 7: Box Plot of Average_Play_Time

(Source: Self-Created)

The image shows a boxplot of the 'Average_Play_Time' column. The distribution is right skewed with several outliers above the upper quartile. The median play time is around 50 units, with most values falling between 25 and 100.

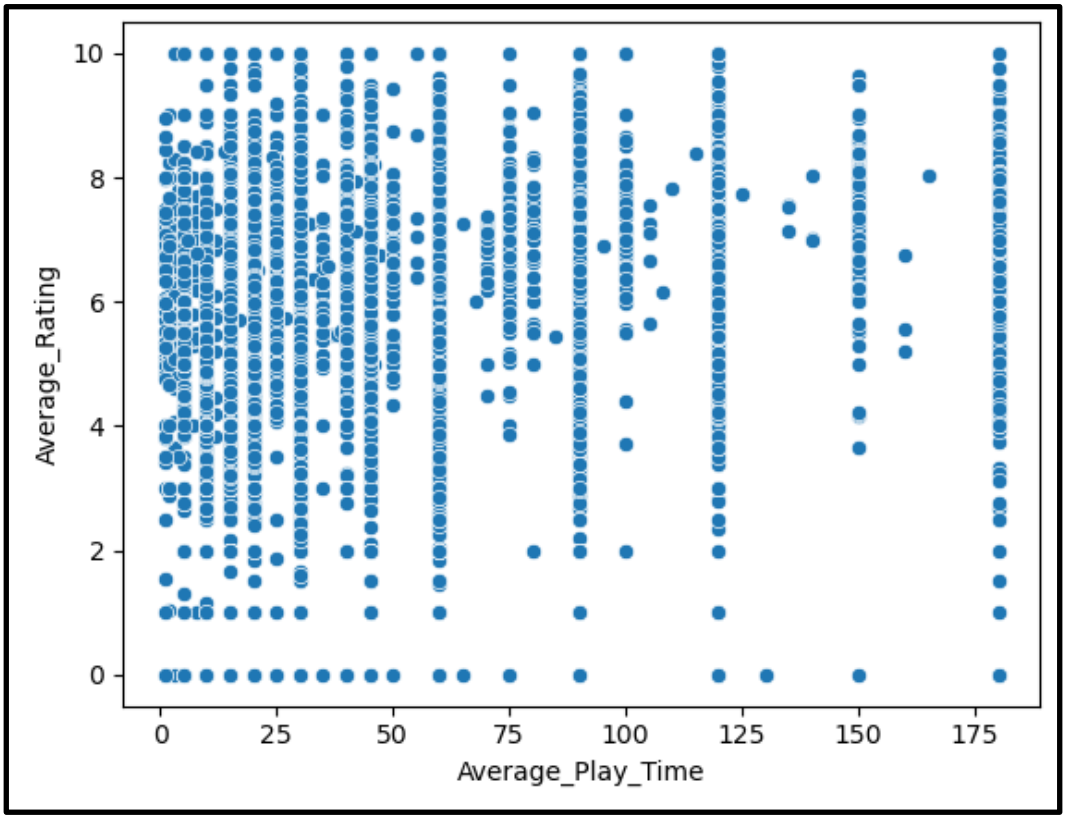


Figure 8: Scatter Plot

(Source: Own-Creation)

The image shows a scatter plot of Average Play Time vs Average Rating. There seems to be a weak positive correlation between the two variables, with a wide range of ratings for different play times. Most data points are clustered in the lower left corner, indicating a higher frequency of shorter play times with lower ratings.

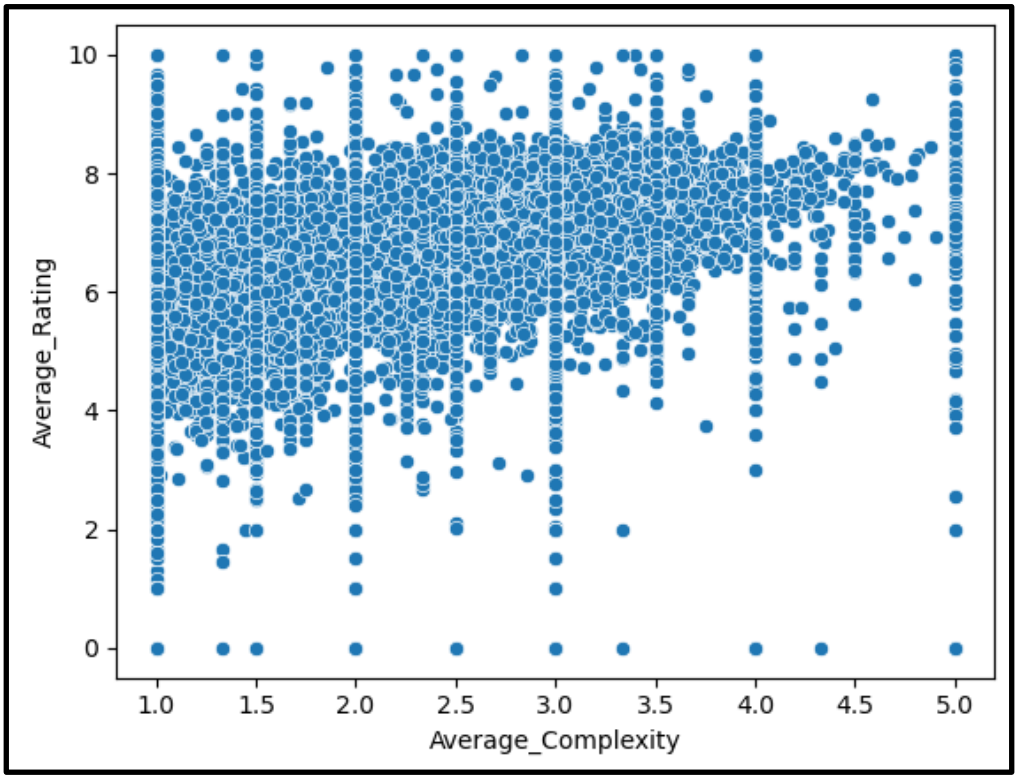


Figure 9: Scatter Plot of Average_Rating vs Average_Complexity

(Source: Self-Made)

The image shows a scatter plot of Average Complexity vs Average Rating. There appears to be a weak positive relationship between the two variables, with a wide range of ratings for different complexities.

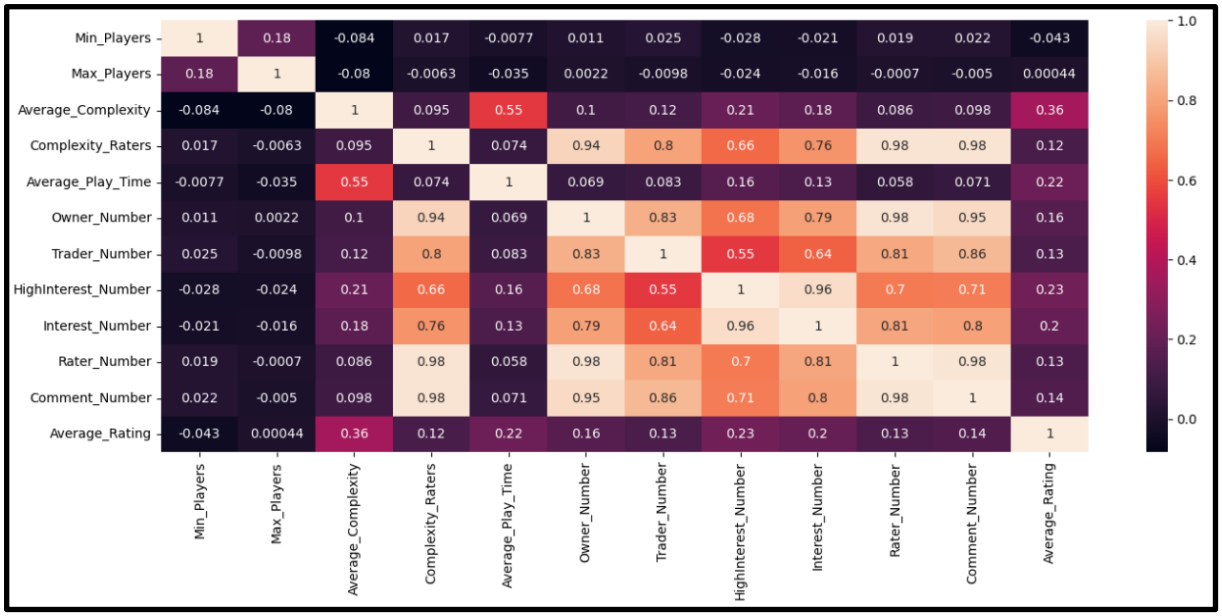


Figure 10: Heatmap (Correlation Matrix)

(Source: Self-Developed)

The image shows a correlation matrix between various features. It reveals strong positive correlations between 'Complexity_Raters', 'Owner_Number', 'Trader_Number', 'HighInterest_Number', 'Interest_Number', and 'Rater_Number'. 'Average_Complexity' and 'Average_Play_Time' also exhibit a moderate positive correlation. 'Average_Rating' appears to have minimal correlation with most features.

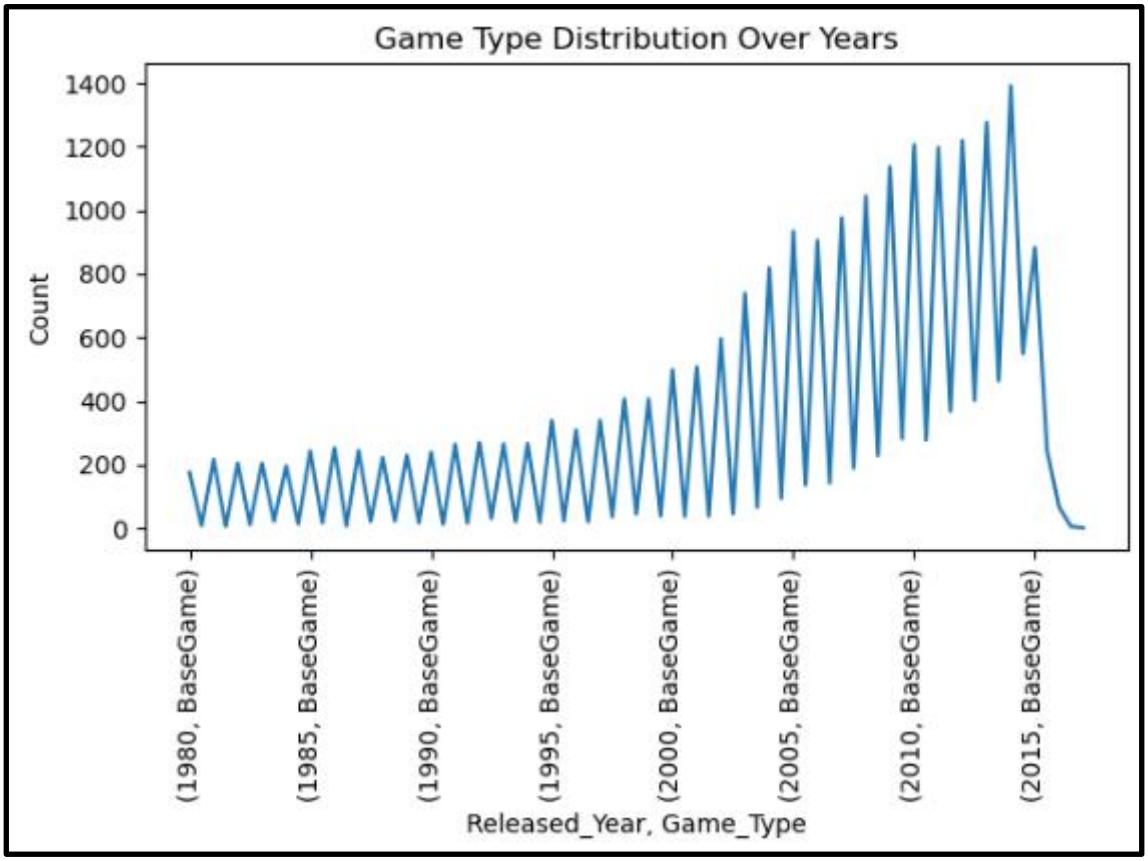


Figure 11: Line plot of Released_Year, Game_Type

(Source: Self-Created)

The image shows a line plot visualizing the count of games released per year, categorized by game type. The plot reveals an increasing trend in game releases over the years, with a more pronounced rise in BaseGame releases compared to PremiumGame.

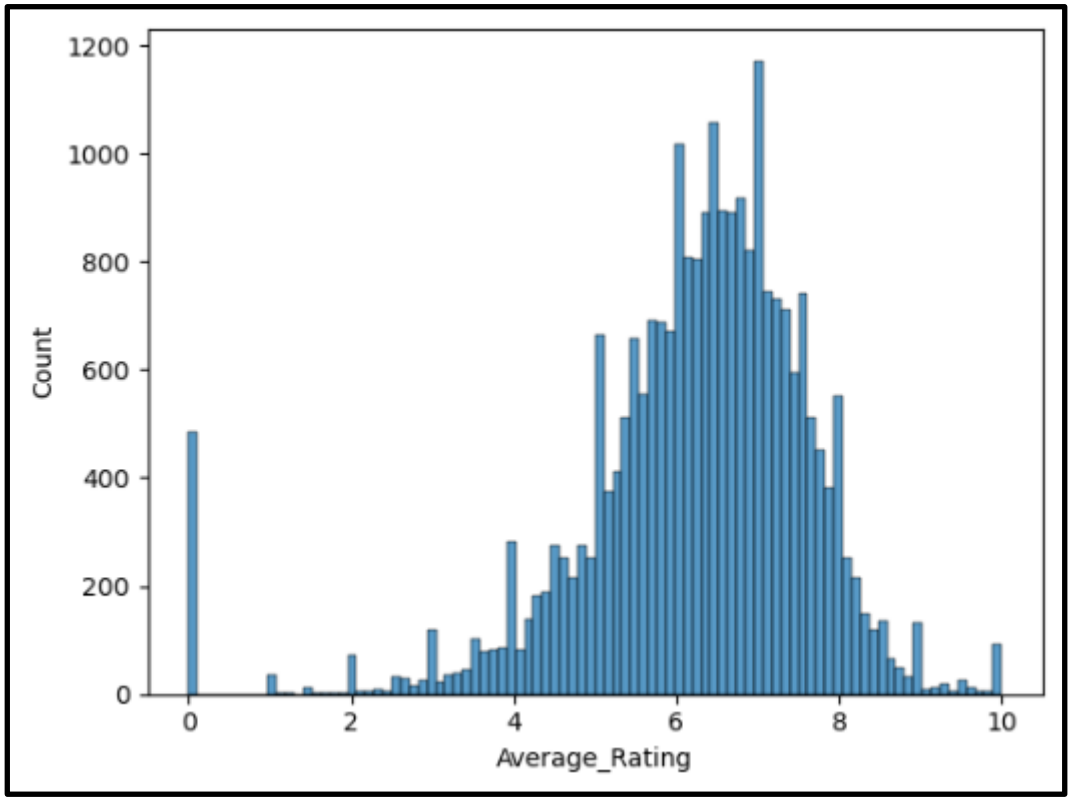


Figure 12: Histogram of Average_Rating

(Source: Own-Creation)

The image shows a histogram of average ratings. The distribution is right skewed with a peak around 7-8, indicating most games have average ratings in this range. There are fewer games with very low or very high average ratings.

Insights Gained

Initial exploration of this data returns information about popular game types, the trends of game releases over the years, age categories, and player requirements for different games. Average playtime analysis can give outliers that may require further examination. This knowledge of the relationship among playtime, complexity, and ratings provides a base from which predictive

models can be engendered. These battle-tested insights provide guidance not only on the choice of features but also on how to design machine-learning models for predicting game ratings at very high accuracy..

3. The Machine Learning Approach

Model Selection

The correct choice for the appropriate algorithm of supervised machine learning is very important in developing an effective predictive model. For the analysis in this case, algorithms like Linear Regression, Decision Trees, and Random Forests have been considered. Since Linear Regression is simple and highly interpretable, it would be good to predict continuous variables starting with a game rating variable. Decision Trees are flexible in terms of handling numerical and categorical data and capturing nonlinear relationships . One of the ensemble methods, random forests, improves the accuracy and robustness of a prediction by grouping many decision trees together.

Model Development

Listed below are some of the critical stages in the process of model development. Firstly, the dataset is divided into train and test sets for estimating the quality of the model. The model will then be trained using this training set. Using this testing set, the model's generalization ability is tested. Hyper-parameters are tuned to bring out the best from the model's

performance by changing various parameters such as the depth of decision trees or the number of trees within a Random Forest. Cross-validation methods ensure the strength of the model and prevent overfitting

Performance Metrics

Model performance metrics include mean absolute error, squared error, and r squared. The MAE computes the average absolute difference between predicted and true ratings, so it does give a sense of the magnitude of prediction errors . Squaring the differences before taking their average, MSE gives another perspective, since larger errors are penalized more heavily. It indicates that R^2 is supposed to show the proportion of variance in a dependent variable explained by the model, thus showing its explanatory power.

4. The Model and Performance Metrics

Model Description

The development ends by choosing the best model, which will be used with its parameters founded on performance metrics. For example, a Random Forest model is chosen because it can handle complex relations and provide highly accurate predictions Details of the structure are given, such as the number of trees and the depth of each tree. This random forest model will

work fine in this analysis because it avoids overfitting and can support numerical as well as categorical variables.

Performance Evaluation

Model performance is also discussed in terms of accuracy and error metrics. Both the MAE and MSE provide information about the average prediction error, while R^2 gives returns regarding how appropriate the model is to explain the variance in game ratings . The higher the value of R^2, the more of the variance in ratings explained by the model hence, it will be reliable. These metrics quantify both the strengths and weaknesses of the model and, hence, its practical applicability in the prediction of game ratings.

5. Discussion of the Model: Pros and Cons

Pros

One of the key benefits of this chosen model, such as Random Forest, is that it provides high accuracy in predicting game ratings and reliable insights for strategic decision-making. The robustness of the model in handling various input features and their interactions greatly enhances its predictive power. In addition, due to its flexibility, it will be relevant over time and scale gracefully with increasing data and additional features added in the future. This is further simplified by the fact that the model can process numerical data as well as categorical data without intensive preprocessing.

Cons

However, there are also some limitations of the Random Forest model. First, the increased complexity makes it more challenging to interpret than less complex models such as Linear Regression; additional techniques, like feature importance analysis, may be necessary in explaining how the contribution of individual features affects the predictions. Moreover, both training and making predictions with this model can be more computationally expensive. Therefore, especially for large input data, sufficient computational resources should be at one's disposal. These factors need to be considered when implementing and maintaining the model in a production environment.

6. Business solutions and recommendations

Implementation

There are several strategic initiatives that, based on the gained insights, can be implemented by PQC. Targeted marketing strategies for promoting games with a high predicted rating will increase user engagement and satisfaction. Enhanced game development through implementation based on features that positively influence ratings can result in more tolerantly accepted games with increased loyalty. Improvement in user engagement driven by personalized recommendations and tailored user experiences informed by the predictions of the model will further increase satisfaction and retention.

Recommendations

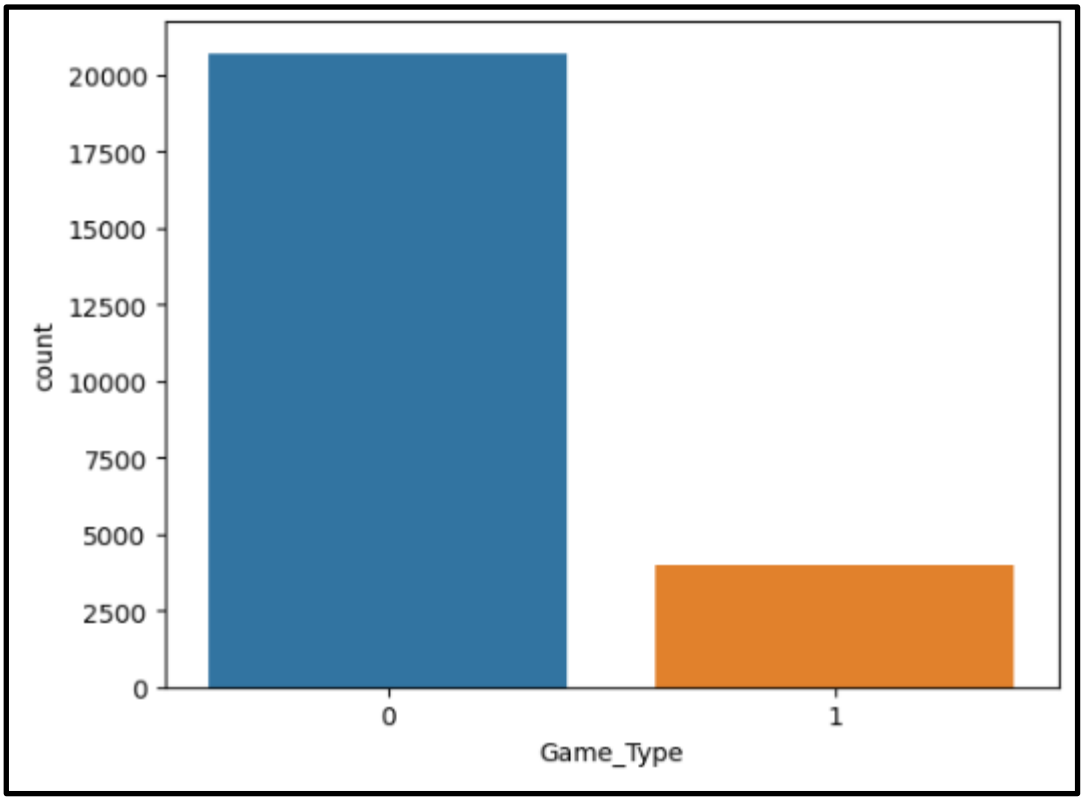
PQC must not let up on ensuring data quality but instead continue improving data collection to ensure this model remains effective. Relevance and accuracy updated data will keep the model current. User feedback is going to be important in giving insights into user preferences to enhance predictive power. Indeed, taken together, these recommendations show a pathway through which PQC can finally tap into the full potential of this machine learning model for growth in both business and user satisfaction.

Conclusion

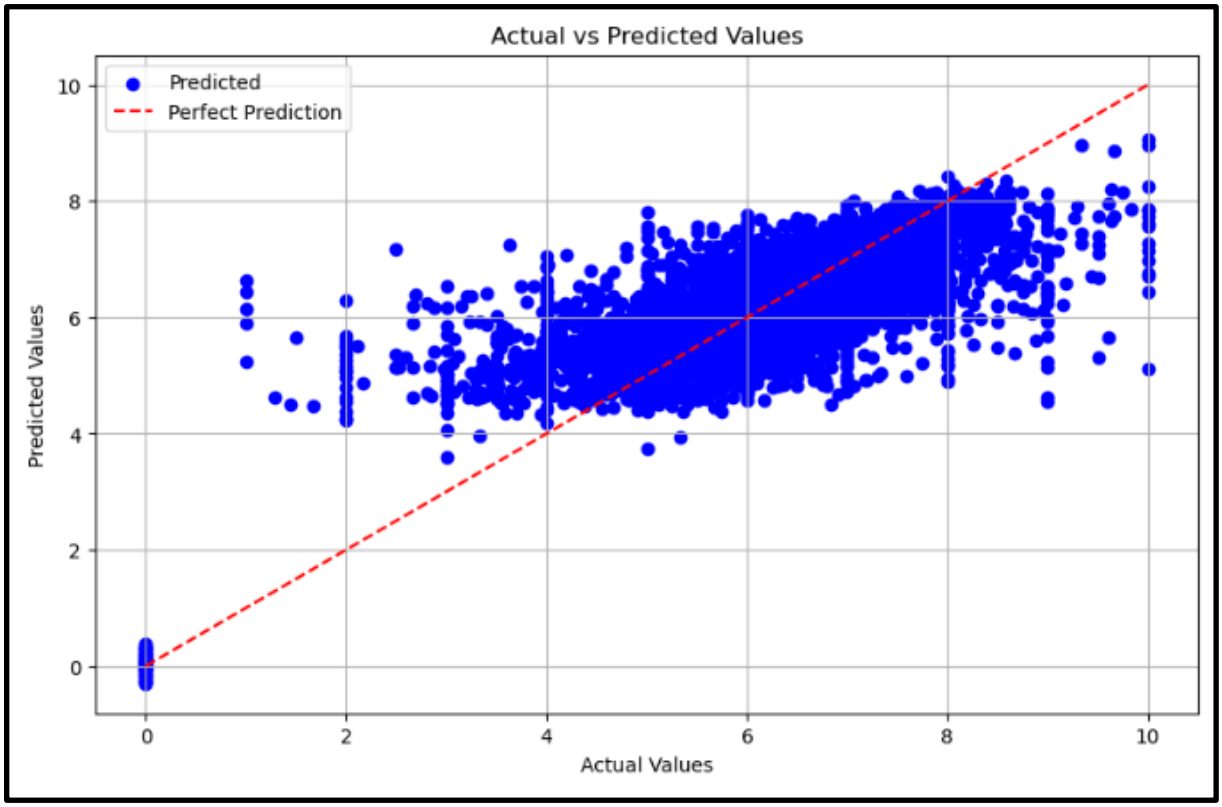
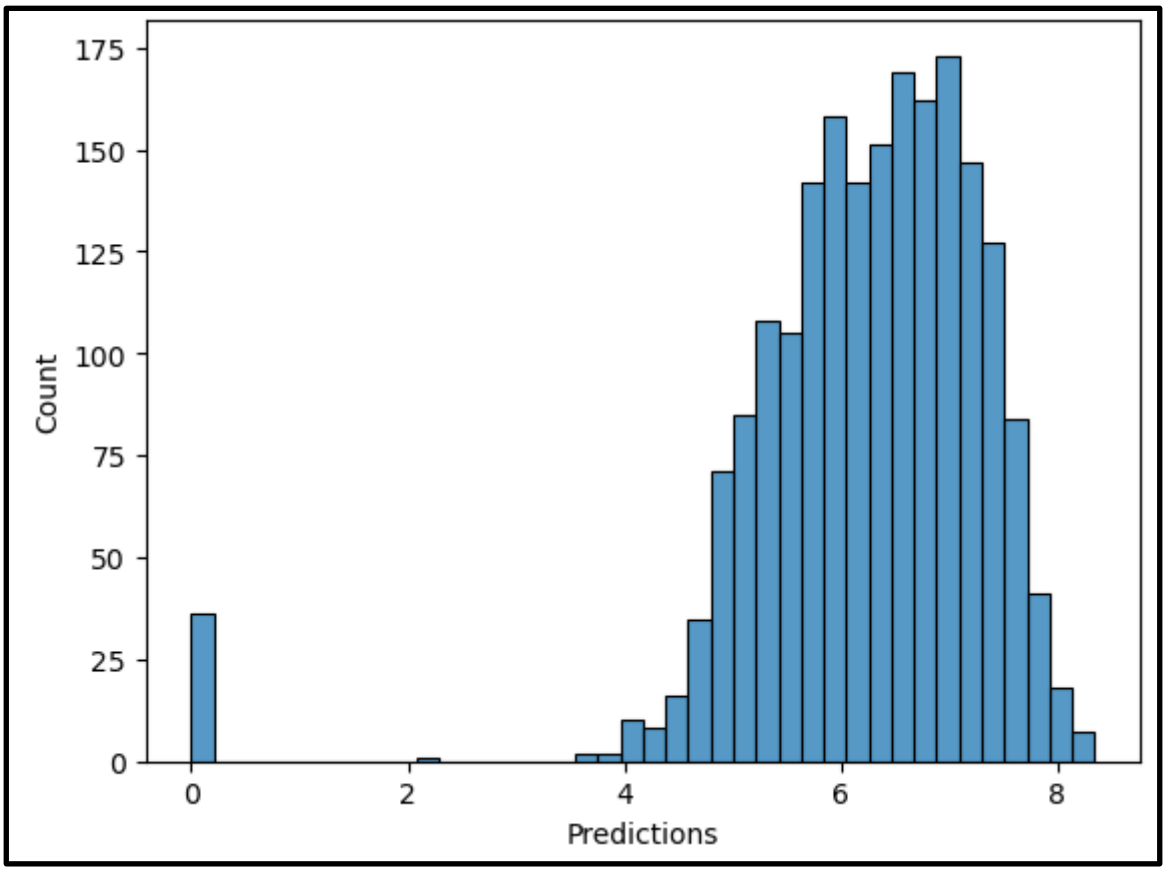
By using supervised machine learning, PQC can unlock valuable knowledge about variables at play with respect to rating games. This data-driven approach supports strategic decisions that embody better user experience, more game acquisitions, and efficient resource allocation. Applied as a result, this developed machine learning model will be a robust and scalable solution toward predicting game ratings, hence escalating PQC with improved service offerings and advantages over the competition in the online gaming community.

Appendices

Appendix 1: Count Plot of Game_Type



Appendix 2: Scatter Plot between Actual values vs Predicted values



Appendix 3: Histogram of Predicted values